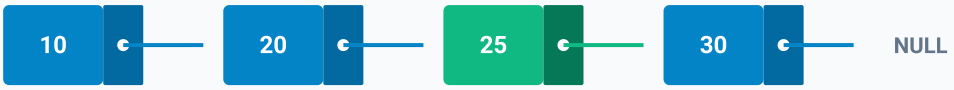
Data Structures Lab Guide

Comprehensive Implementation Reference: Linked Lists & Stacks in C++

Topics: Singly, Doubly & Circular Lists, Stacks Language: C++ Level: Data Structures & Algorithms

1. SINGLY LINKED LIST (APPEND & POSITION INSERT)

A Singly Linked List consists of dynamic nodes where each node stores data and a pointer to the next node. Below is an object-oriented class implementation featuring sequential append and positional insertion.




```
// LinkedList01.cpp - Singly Linked List Implementation #include <iostream> using namespa
```

Console Output:
Original List: 10 20 30 40
After Insert: 10 20 25 30 40

2. DOUBLY LINKED LIST IMPLEMENTATIONS

A Doubly Linked List contains nodes with dual pointers (`prev` and `next`), allowing bidirectional traversal. Below are both direct dynamic allocation and fully dynamic class-based solutions.

Manual Two-Node Construct

Direct allocation using pointers. Explicitly connects `first->next = second` and `second->prev = first`.

Encapsulated Class Driver

Dynamic tail navigation via `append()` loop, setting full forward and reverse linkage automatically.

2.1 Manual Pointer Linking (Double_LinkedList02.cpp)

```
#include <iostream> using namespace std; struct Node { int data; Node* prev; Node* next;
```

2.2 Dynamic Class Implementation (Double_LinkedList.cpp)

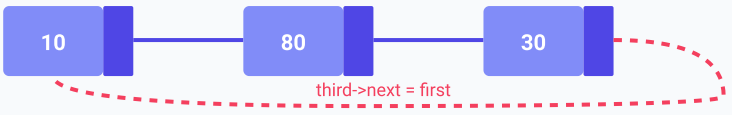
```
class DoublyLinkedList { private: Node* head; public: DoublyLinkedList() { head = NULL; }
```

Console Output:
Doubly Linked List: 10 20 30

3. CIRCULAR LINKED LIST & BASIC ARRAY STACK

This section explores circular linkage where the final node references the head node, alongside simple fixed-size array-based stacks.

3.1 Circular Linked List (Circular_LinkedList.cpp)




```
// Circular linkage manually linking back to start Node* first = new Node; Node* second =
```

3.2 Array Stack Operations (Stack01.cpp & Stack02.cpp)

Arrays provide LIFO (Last-In, First-Out) stack operations using an index variable tracking the top elements.

```
// Stack02.cpp - Array-based Stack with Overflow/Underflow Bounds Check int stackArray[5]
```

Operation	Time Complexity	Space Complexity	Boundary Condition
Array Push	O(1)	O(1)	Overflow when `top == size - 1`
Array Pop	O(1)	O(1)	Underflow when `top == -1`

4. COMPREHENSIVE STACK (LINKED LIST-BASED)

A dynamically sized stack implemented via dynamic Node allocation. Avoids fixed-array size limitations and manages memory via explicit destructor cleanup.

```
// Lab_Task.cpp - Complete Dynamic Stack Implementation #include <iostream> using namespa
```

Program Output:
10 pushed into stack.
20 pushed into stack.
30 pushed into stack.

Top element: 30 | Stack size: 3
30 popped from stack.
Top element after pop: 20 | Stack size: 2
Stack is not empty.